SDA Quiz 4 - 4A

10/10

Roll No: 24L-0727 Name: Sadia Asif

Q1. Your engineering team maintains a critical microservice that pulls data from a third-party financial API using a core RestApiFetcher component. Production has been experiencing intermittent slowdowns. To diagnose this, the Site Reliability Engineering (SRE) team has issued a new mandate: every API request must be timed, and the execution duration must be logged to the telemetry system.

Because the RestApiFetcher is part of a locked, third-party vendor SDK, you are strictly forbidden from modifying its source code. You must also avoid creating static subclasses, as the team plans to add more dynamic behaviors (like retry loops and caching) to the fetcher in the next sprint.

(Note: For the execution timing, you may use pseudo-code like start_timer() and stop_timer() inside your function. Focus your effort on providing the correct Object-Oriented structure and C++ class relationships).

```
#include <iostream>
#include <string>

// The base interface
class IApiFetcher {
public:
    virtual ~IApiFetcher() = default;
    virtual void FetchData(const std::string& endpoint) = 0;
};

// The concrete component (Locked Vendor Code)
class RestApiFetcher : public IApiFetcher {
public:
    void FetchData(const std::string& endpoint) override {
        // Simulating a network call
        std::cout << "Connecting to network...\n";
        std::cout << "Downloading payload from: " << endpoint << "\n";
    }
};

// Client execution
int main() {
    IApiFetcher* fetcher = new RestApiFetcher();
    fetcher->FetchData("https://api.finance.com/v1/markets");

    delete fetcher;
    return 0;
}
```

liner adder here. written on back at the end.

fetcher = new telemetary(fetcher);

Task

Apply a design pattern to introduce the telemetry/timing requirement.

1. Write the C++ code for your new decorator classes.
2. Update the main() client function to demonstrate how the telemetry behavior is dynamically attached to the standard fetcher using pointers.